

CIS241

System-Level Programming and Utilities

C - Makefiles!

Erik Fredericks, frederer@gvsu.edu

Fall 2025

Based on material provided by Erin Carrier, Austin Ferguson, and Katherine Bowers

MAKE IT SO

make

Let's avoid manually creating intermediate build files

Remember (assuming these files exist...):

```
gcc main.c cool_math.c vector.c
```

Or

```
gcc -c vector.c -o vector.o ;
```

```
gcc -c cool_math.c -o cool_math.o
```

```
gcc main.c vector.o cool_math.o
```

`make` (Gnu variety)

- Defines compilation and linker rules
 - Recursively compiles files as necessary

1. Create a makefile (named `Makefile`)

2. Run `make` (just that word!)

- Alternatively, run `make rule`, where `rule` is defined by you

Creating a Makefile

Makefile is a list of rules.

Format of a rule:

```
target: prereqs ...  
    recipe  
    ...
```

target - Name of file to create or action to perform

prereqs - Files needed to create / perform target

recipe - Sequence of commands to perform rule

Note, recipes line start with a *tab*, not spaces!

- Ctrl + V then Tab in Vim (if you use spaces via expandtab)

Example (simple)

To compile our example from earlier

```
default: main.c vector.c cool_math.c  
        gcc -o program main.c vector.c cool_math.c
```

Note that running make will run the "default" rule, or the first in the file if default doesn't exist

Example (advanced)

To compile our example from earlier

```
default: program
program: main.c vector.o cool_math.o
        gcc -o program main.c vector.o cool_math.o
vector.o: vector.c
        gcc -o vector.o -c vector.c
cool_math.o: cool_math.c
        gcc -o cool_math.o -c cool_math.c
```

Future compilation will only recompile what's needed! (uses file modified dates/times)

Making changes easier

What if we want to change compilers later?

- We don't want to edit every line...
- We can use variables!

```
CC = gcc
CFLAGS = -Wall
program: main.c vector.o
        $(CC) $(CFLAGS) -o program main.c vector.o
```

Note we access variables with `$(var)`

- This is different from bash scripts!

Special tokens

Make gives us some handy tools:

`$@` - expands to the name of the current rule

`$<` - expands to name of first prereq

`$$` - expands to space-separated prereq list

`%` - Matches a pattern that we can use later in line

```
%.o: %.c
    gcc -o $@ -c $<
```

This generalizes our `vector.o` and `cool_math.o` from our example!

https://web.mit.edu/gnu/doc/html/make_4.html

Comments?

Start with `#` - just like bash

Action rules

Some rules don't create a file, they perform an action!

The classic example is `clean`:

```
clean:  
    rm program vector.o cool_math.o
```

We can run this with `make clean`

Another common example is the `install` rule

Why `make`?

Allows us (and others who use our code) to easily compile

Why use `make` over a bash script?

- The recursive prereq lookup
- The date-checking to prevent us from recompiling files that haven't changed

Newer approach: `cmake`